



Gymnázium J. K. Tyla

DarkFactory: Umělá inteligence v praxi

- Agentické a harnessové inženýrství

**DarkFactory: Artificial Intelligence in Practice – Agentic
and Harness Engineering**

ODBORNÁ PRÁCE | THESIS

Autor práce | Author: Rozsah práce | Extent: 1384 slov | words / 9432 znaků | characters
Patrik Marius, 4.D
Vedoucí práce | Supervisor: Michal Dočekal

Prohlášení | Declaration

Prohlašuji, že jsem tuto studentskou odbornou práci vypracoval/a samostatně pod dohledem vedoucího uvedeného na první straně. Všechny použité zdroje jsou uvedeny v seznamu zdrojů a informace z nich získané jsou v textu řádně označeny odkazem na zdroj. Souhlasím s tím, aby tištěná forma práce byla uchována na Gymnázium J. K. Tyla a tam používána jako tištěný zdroj např. pro další studentské práce či pro prezentaci vzdělávání na GJKT.

I declare that I prepared this specialized thesis independently under the supervision of the supervisor named on the title page. All sources used are listed in the bibliography and information derived from them is cited in the text. I agree that the printed version may be archived at Gymnázium J. K. Tyla and used there as a reference for future student work or for presenting education at GJKT.

V Hradci Králové dne | In Hradci Králové onPodpis autora práce
| Author signature:

Anotace | Annotation

[CZ] Tato odborná práce se zabývá principy agentického inženýrství (*agentic engineering*): efektivními inženýrskými praktikami pro vývoj pomocí umělé inteligence prostřednictvím agentických systémů a architekturou těchto systémů. Praktickým přínosem práce je návrh a implementace systému DarkFactory — agentního harnessu instalovatelného jako aplikace pro platformu GitHub (GitHub App). Systém usiluje o maximální možnou míru automatizace vývojového cyklu od interpretace požadavků v GitHub Issues, přes plánování, až po vývoj kódu a sloučení změn. Práce reflektuje, že současné agentní systémy nelze vnímat jako plně autonomní: jazykové modely vyžadují deterministické mantinely, správu kontextu a zapojení člověka (*Human-in-the-loop*).

[EN] This thesis examines the principles of agentic engineering: effective engineering practices for development with artificial intelligence through agentic systems and the architecture of these systems. The practical contribution of the thesis is the design and implementation of DarkFactory — an agentic harness installable as a GitHub App. The system aims to maximize automation of the development lifecycle, from interpreting requirements in GitHub Issues, through planning, to code development and merging changes. The thesis reflects that current agentic systems cannot be regarded as fully autonomous: language models require deterministic guardrails, context management, and human involvement (*Human-in-the-loop*).

Klíčová slova | Keywords

Agent, Agentické inženýrství [Agentic Engineering] , Agentní harness [Agent Harness] , Chatbot, Dovednosti [Skills] , Git, GitHub, Jazykový model [Large Language Model] (LLM) , Rozšíření [Plugins] , Smyčka ReAct [ReAct Loop] (Agent Loop) , Vektorová reprezentace [Embedding]

Obsah | Contents

Prohlášení Declaration	2
Anotace Annotation	3
Klíčová slova Keywords	4
1 Úvod	8
1.1 Motivace a vymezení problému	8
1.2 Cíl práce a výzkumné otázky	8
1.2.1 Hlavní cíl	8
1.2.2 Dílčí cíle	8
1.2.3 Výzkumné otázky	8
1.3 Metodika práce	8
2 Agentické AI: Vymezení konceptů - Teoretická část	9
2.1 Vývojové prostředí a praxe	9
2.1.1 Správa verzí [Version Control]	9
2.1.2 Git a GitHub	9
2.1.3 Větvě (Branches)	9
2.1.4 Pull Request	9
2.1.5 Slučování změn (Commit and Merge)	9
2.1.6 Plánování [Planning]	9
2.1.7 Kontinuální integrace (CI a GitHub Actions)	9
2.1.8 Požadované kontroly (Required Checks)	9
2.2 Large Language Model (LLM) , chatboti a agenti	9
2.2.1 Úvod	9
2.2.2 Agent vs. Chatbot	10
2.2.3 Tokeny, tokenizace a Vektorová reprezentace [Embedding]	10
2.3 Agentické inženýrství [Agentic Engineering] a Agent Harness	11
2.3.1 Úvod	11
2.3.2 Smyčka ReAct [ReAct Loop] (Agent Loop)	11
2.3.3 Vyvolávání nástrojů [Tool Calling]	12
2.3.4 Dovednosti [Skills]	12
2.3.5 Škálování: Multiagentní systémy (Subagenti) a grafy (DAG workflows) [Scaling: Multiagent Systems (Subagents) and DAG Workflows (Graphs)]	13
3 DarkFactory: Architektura harnessu - Praktická část	14
3.1 Git a GitHub	14
4 Výsledky a diskuse	15

5 Závěr	15
Rejstřík Index	15
Seznam příloh List of appendices	24
Seznam zdrojů References	25

Seznam obrázků a tabulek | List of figures and tables

Obrázek 1	Architektura autonomní ReAct smyčky (Reasoning + Acting) a tok dat mezi uživatelem, kontextem, modelem a výkonným prostředím.	12
-----------	--	----



KAPITOLA | CHAPTER 1

Úvod

1.1 Motivace a vymezení problému

Ústřední inženýrská otázka této práce proto nespočívá v tom, zda jazykový model dokáže napsat fragment kódu. Zkoumáme, jaká kontrolní a dozorčí architektura — značovaná jako *Agent Harness* — musí model obklopovat, aby bylo možné jeho výstupům v produkčním repozitáři spolehlivě důvěřovat a dosáhnout vysoké míry autonomie se zachováním lidského dohledu.

1.2 Cíl práce a výzkumné otázky

1.2.1 Hlavní cíl

Vymezit teoretické principy agentického inženýrství (*agentic engineering*) a navrhnout modulární architekturu agent harnessu pro automatizovaný vývoj softwaru se zachováním lidského dohledu v klíčových rozhodovacích bodech.

1.2.2 Dílčí cíle

- Vymezit infrastrukturu pro správu verzí (Git, GitHub a kontinuální integraci).
- Analyzovat limity velkých jazykových modelů (dynamiku kontextového okna, jev Context Rot, ztrátovou kompresi a sémantický posun).
- Navrhnout architekturu agent harnessu zahrnující nástrojové smyčky (ReAct), bezpečnostní pískoviště a hierarchickou orchestraci subagentů.
- Formalizovat mechanismy zapojení člověka do smyčky (*Human-in-the-loop*), schvalovací brány a protokol revizních značek pro dohled nad textovými výstupy.

1.2.3 Výzkumné otázky

1.3 Metodika práce

Práce má teoreticko-architektonický a inženýrský charakter. Vzhledem k dynamickému vývoji v oblasti autonomního softwarového vývoje práce důsledně zachovává a integruje zavedené anglické odborné názvy (např. *harness*, *pull request*, *agent loop*, *prompt engineering*, *skills* či *context rot*). Použití této terminologie je integrální součástí práce, neboť tyto anglické pojmy představují de facto celosvětové průmyslové standardy (*industry standards*), jejichž doslovný český překlad by byl nejednoznačný, zavádějící či v rozporu s běžnou inženýrskou praxí.

Postup práce sleduje strukturu inženýrského cyklu:

- 1. Analýza konceptu: Systematické zmapování limitů autoregresivních modelů, dynamiky kontextového okna, jevu Context Rot a rozhraní nástrojů.
- 2. Návrh architektury: Formulace modulárního modelu agent harnessu, správy stavu, exekučního pískoviště, bezpečnostních pojistek a orchestrace subagentů.
- 3. Kritické zhodnocení: Porovnání navržených principů s volnými agentními smyčkami a vymezení provozních limitů autonomního inženýrství.



KAPITOLA | CHAPTER 2

Agentické AI: Vymezení konceptů - Teoretická část

2.1 Vývojové prostředí a praxe

2.1.1 Správa verzí [Version Control]

2.1.2 Git a GitHub

Pro autonomní vývoj softwaru je spolehlivá správa verzí naprosto nezbytným základem. Jazykové modely generují kód na základě statistické pravděpodobnosti, a proto se nevyhnutelně dopouštějí chyb, logických přehmatů či regresí. Verzovací systém vytváří bezpečné a deterministické prostředí, v němž lze každou úpravu zaznamenat, otestovat a v případě selhání kdykoliv vrátit zpět k funkčnímu stavu. Namísto teoretických abstrakcí práce přímo využívá distribuovaný systém **Git** v kombinaci s platformou **GitHub**.

Klíčové komponenty infrastruktury zahrnují:

- Distribuovaný systém Git (1): Ukládá kompletní historii projektu v podobě jednotlivých revizí (*commitů*). Vývojář i agent pracují s plnou lokální kopií repozitáře, což umožňuje provádět změny, přepínat větve a spouštět lokální testy zcela nezávisle na síťovém připojení.
- Platforma GitHub: Slouží jako centrální bod pro sdílení kódu, týmovou koordinaci a automatizaci:
 - Zadávání a sledování úkolů (Issues): Strukturovaná textová zadání požadavků a hlášení chyb, která agentovi slouží jako výchozí specifikace úlohy.
 - Revize změn (Pull Requests): Uživatelské rozhraní pro přehledné zobrazení diffu, diskusi nad kódem a formální schválení člověkem.
 - Automatizace (GitHub Actions): Běhové prostředí pro automatické spouštění testů, linterů a překladů při každé události v repozitáři.

Agent v tomto pojetí nevystupuje jako černá skříňka s proprietárním protokolem, nýbrž jako standardní přispěvatel, který plně respektuje běžné vývojářské zvyklosti a nástroje.

2.1.3 Větve (Branches)

2.1.4 Pull Request

2.1.5 Slučování změn (Commit and Merge)

2.1.6 Plánování [Planning]

2.1.7 Kontinuální integrace (CI a GitHub Actions)

2.1.8 Požadované kontroly (Required Checks)

2.2 Large Language Model (LLM) , chatboti a agenti

2.2.1 Úvod

V agentickém softwarovém inženýrství vystupuje velký jazykový model (LLM) jako stochastické kognitivní jádro celého systému. Z hlediska vnitřní architektury se jedná o dekodérový

transformer (*Decoder-only*), jehož typickými představiteli jsou moderní modely řad Claude, GPT či DeepSeek (2). Role modelu nespočívá ve vystupování jako vševědoucí orákulum se spolehlivou znalostí okolního světa, nýbrž jako pokročilý generátor hypotéz, kódu a strukturovaných volání nástrojů řízený obdrženým kontextem.

Základní principy fungování modelu zahrnují:

- Autoregresivní predikce: Model zpracovává zadanou sekvenci textu a na jejím základě iterativně předpovídá nejpravděpodobnější následující symboly (tokeny).
- Stochastická povaha: Vzhledem k pravděpodobnostnímu vzorkování může model na totožný vstup reagovat mírně odlišně, což vyžaduje deterministické mantinely v nadřazeném agent harnessu.

Pro efektivní nasazení modelu do vývojového cyklu je nezbytné porozumět způsobu, jakým reprezentuje informace a jaké fyzické limity vymezují jeho operační paměť.

2.2.2 Agent vs. Chatbot

Chatbot★ — [cz] Systém založený na jazykovém modelu určený primárně k textové interakci s uživatelem; odpovídá na jednotlivé požadavky, ale sám o sobě nedisponuje autonomní prováděcí smyčkou ani nástroji pro samostatnou modifikaci okolního prostředí.. **Agent**★ — [cz] Softwarový systém řízený jazykovým modelem a vybavený nástroji, který samostatně plánuje, vnímá stav prostředí a provádí víceukrokové akce směřující k dosažení zadaného inženýrského cíle.. Rozdíl mezi nimi nespočívá v odlišném jazykovém modelu, ale v architektuře jeho zapojení do pracovního prostředí.

Srovnání obou přístupů:

- Konverzační chatbot:
 - Reaguje pouze na přímé textové výzvy v uzavřeném okně chatu.
 - Nemá přímý přístup k souborovému systému ani k nástrojům operačního systému.
 - Uživatel musí navržený kód ručně zkopírovat, vložit do projektu a otestovat.
- Autonomní agent:
 - Je vybaven sadou výkonných nástrojů (*tools*) pro práci s repositářem.
 - Aktivně prozkoumává soubory, modifikuje zdrojový kód, spouští testy a interpretuje jejich návratové kódy.
 - Funguje v autonomní prováděcí smyčce, v níž iterativně reaguje na reálnou odezvu vývojového prostředí.

2.2.3 Tokeny, tokenizace a Vektorová reprezentace [Embedding]

Jazykový model nepracuje přímo se znaky ani slovy v lidském slova smyslu. Vstupní text je nejprve deterministickým algoritmem převeden na číselné reprezentace, se kterými následně počítají maticové vrstvy neuronové sítě.

Tento proces zahrnuje následující pojmy:

- Tokeny a tokenizér: Token představuje základní diskrétní jednotku (celé slovo, slabiku či fragment znaků). Převod mezi textem a posloupností číselných tokenů zajišťuje tokenizér (nejčastěji na bázi algoritmu Byte Pair Encoding, BPE).
- **Vektorová reprezentace [Embedding]** ★ — [cz] Vícerozměrná vektorová reprezentace tokenů nebo jiných dat, v níž numerické vztahy mezi vektory zachycují užitečné

sémantické vztahy mezi reprezentacemi. (např. vektorová analogie král — muž + žena $\approx$ královna).

- Jazyková asymetrie tokenizace: Vzhledem k trénovacím datům optimalizovaným primárně pro angličtinu spotřebovávají flektivní jazyky s bohatou diakritikou (včetně češtiny) 2× až 3× více tokenů pro vyjádření téhož významu.

Z inženýrského hlediska je proto žádoucí vést systémové prompty, technické plány i komunikaci mezi nástroji v angličtině, aby se šetřila kapacita kontextu a snížila latence inference.

2.3 Agentické inženýrství [**Agentic Engineering**] a Agent Harness

2.3.1 Úvod

V terminologii agentického inženýrství používá tato práce pojem **Agentní harness** [**Agent Harness**] *. [CZ] Agentní harness — aplikační a orchestrační vrstva obklopující inferenční jádro modelu, která zajišťuje běhové prostředí nástrojů, dynamickou správu kontextového okna, bezpečnostní mantinely, práci se stavem a deterministické řízení životního cyklu požadavku.. Samotné inferenční jádro provádí výhradně matematické maticové operace nad zadanými váhami a vektory tokenů; veškerou orchestraci, práci se soubory a řízení bezpečnosti zajišťuje harness.

Ústřední komponentou a hlavní prováděcí funkcí, která v architektuře harnessu řídí samotný běh a iterativní koordinaci agenta v reálném vývojovém prostředí, je **Smyčka ReAct** [**ReAct Loop**] (**Agent Loop**) *. [CZ] Iterativní prováděcí cyklus autonomního agenta založený na vzoru ReAct (Reasoning + Acting), v němž model střídavě uvažuje, volá nástroje a vyhodnocuje pozorování z běhového prostředí..

2.3.2 Smyčka ReAct [**ReAct Loop**] (**Agent Loop**)

Agentní smyčka (*Agent Loop*) představuje výkonné jádro celého agent harnessu. Zatímco pasivní konverzační chatbot jednorázově odpoví na uživatelský dotaz a čeká na další vstup, agentní smyčka autonomně udržuje kontinuální iterativní proces, v němž harness opakovaně vyhodnocuje stav repozitáře, volá jazykový model a vykonává požadované systémové akce.

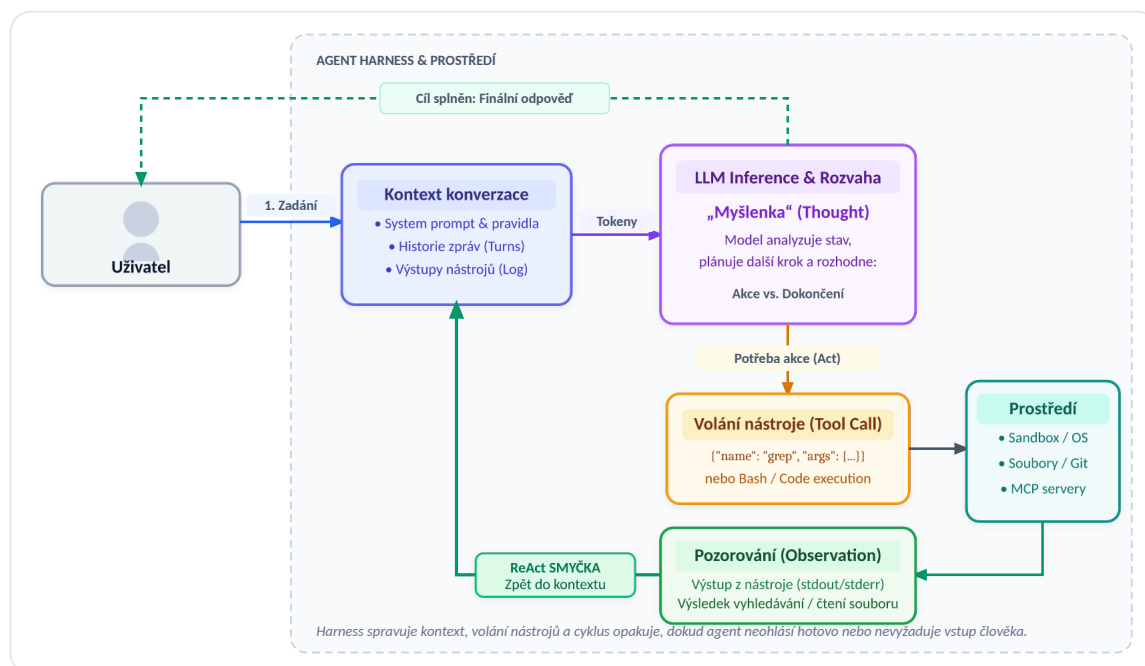
V každé iteraci agentní smyčky harness zajišťuje tyto klíčové funkce:

- Inicializace a správa sezení: Sestavení systémového promptu, dynamická injekce kontextu repozitáře a sledování spotřeby tokenů.
- Běhové prostředí nástrojů: Bezpečné spouštění příkazů v operačním systému a zpětné předávání výstupů modelu.
- Řízení stavových přechodů a vynucování mantinelů: Dohled nad dodržováním procesních pravidel, detekce a zastavení uvíznutých běhů a vynucování lidských schvalovacích bran.

Vnitřní kognitivní krok modelu uvnitř smyčky se řídí operačním vzorem ReAct (*Reasoning + Acting*) (3), který propojuje rozvahu s přímým jednáním. Tento prováděcí cyklus sestává ze čtyř navazujících fází znázorněných na Obrázek 1:

1. Rozvaha (*Thought*): Model vyhodnotí aktuální stav kontextu a formuluje svůj nejbližší záměr.
2. Volání nástroje (*Tool Call*): Emitování strukturovaného požadavku na provedení konkrétní akce s určenými parametry.
3. Vykonání a pozorování (*Observation*): Harness bezpečně provede akci v systému a výstup (výpis souboru či chybovou zprávu) vloží zpět do kontextu.
4. Navazující iterace: Model v dalším tahu analyzuje získanou odezvu a rozhoduje o dalším kroku.

Kvalita a provozní spolehlivost celého systému tak závisí v první řadě na robustnosti architektury harnessu a spolehlivosti jeho agentní smyčky, nikoliv pouze na samotném jazykovém modelu.



Obrázek 1: Architektura autonomní ReAct smyčky (Reasoning + Acting) a tok dat mezi uživatelem, kontextem, modelem a výkonným prostředím.

2.3.3 Vyvolávání nástrojů [Tool Calling]

2.3.4 Dovednosti [Skills]

Se vzrůstající komplexitou úloh nelze veškeré instrukce, skripty a doménové znalosti vkládat do základního systémového promptu. K modulárnímu rozšíření schopností agenta slouží **Dovednosti [Skills]** ★ — [CZ] Znovupoužitelné modulární balíčky instrukcí (typicky definovaných v souboru SKILL.md), procedurálních pravidel a volitelných pomocných skriptů či zdrojů, které harness dynamicky načítá do kontextu agenta podle povahy řešeného úkolu..

Architektura dovedností staví na následujících principech:

- Definiční soubor `SKILL.md`: Dovednost tvoří adresář obsahující definiční soubor se strukturovanou hlavičkou (YAML frontmatter vymezující název a popis role) a detailním návodem k použití.
- Dynamické načítání pro úsporu kontextu: Do výchozího promptu se vloží pouze stručný přehled dostupných dovedností. Kompletní instrukce a skripty se do kontextu načtou až v okamžiku, kdy agent danou dovednost explicitně vyvolá.
- Skripty a záchytné body (*Scripts & Hooks*): Dovednosti mohou obsahovat deterministické skripty pro rutinní transformace kódu a událostní háčky vyvolávané při stavových přechodech harnessu.

Kromě kontextových dovedností využívají pokročilé řídicí architektury také programové **Rozšíření [Plugins]** ★ — [CZ] Rozšíření běžící přímo v prostředí harnessu, která rozšiřují jeho exekuční jádro o specializované systémové adaptéry, ovladače nástrojů a deterministické záchytné body. Zatímco *Skills* fungují jako kontextové procedury a instrukce interpretované modelem, pluginy rozšiřují samotný harness na nativní systémové úrovni.

2.3.5 Škálování: Multiagentní systémy (Subagenti) a grafy (DAG workflows) [Scaling: Multiagent Systems (Subagents) and DAG Workflows (Graphs)]

Monolitická agentní smyčka selhává při řešení komplexních, vícefázových úloh. Pro spolehlivé škálování se v moderních systémech uplatňuje hierarchická dělba práce a formalizace procesu do podoby grafu.



KAPITOLA | CHAPTER 3

DarkFactory: Architektura harnessu - Praktická část

Úvod

3.1 Git a GitHub



KAPITOLA | CHAPTER 4

Výsledky a diskuse



KAPITOLA | CHAPTER 5

Závěr

Rejstřík | Index

[Agent](#)

[Agentické inženýrství \[Agentic Engineering \]](#)

[Agentní harness \[Agent Harness \]](#)

[Chatbot](#)

[Degradace kontextu \[Context Rot \]](#)

[Dovednosti \[Skills \]](#)

[Generování rozšířené vyhledáváním \[Retrieval-Augmented Generation \] \(RAG \)](#)

[Git](#)

[GitHub](#)

[GitHub Actions \(Actions \)](#)

[Inženýrství pracovních grafů \[Workflow-graph Engineering \] \(Graph Engineering \)](#)

[Inženýrství prováděcí smyčky \[Execution-loop Engineering \] \(Loop Engineering \)](#)

[Jazykový model \[Large Language Model \] \(LLM \)](#)

[Kompakce kontextu \[Context Compaction \] \(Compaction \)](#)

[Kontextové inženýrství \[Context Engineering \]](#)

[Kontextové okno \[Context Window \]](#)

[Mezipaměť klíčů a hodnot \[Key-Value Cache \] \(KV Cache \)](#)

[Model Context Protocol \(MCP \)](#)

[Orientovaný acyklický graf \[Directed Acyclic Graph \] \(DAG \)](#)

[Požadavek na sloučení \[Pull Request \]](#)

[Promptové inženýrství \[Prompt Engineering \]](#)

[Průběžná integrace \[Continuous Integration \] \(CI \)](#)

[Rozšíření \[Plugins \]](#)

[Skript \[Script \]](#)

[Sloučení commitů \[Commit Squashing \] \(Squash \)](#)

[Sloučení větví \[Branch Merge \] \(Merge \)](#)

[Smyčka ReAct \[ReAct Loop \] \(Agent Loop \)](#)

[Softwarové inženýrství \[Software Engineering \]](#)

[Softwarový kontejner \[Software Container \] \(Container \)](#)

[Správa verzí \[Version control \]](#)

[Tah interakce \[Interaction Turn \] \(Turn \)](#)

[Token](#)

[Tokenizér \[Tokenizer \]](#)

[Transformerová architektura \[Transformer Architecture \] \(Transformer \)](#)

[Událostní zachytňý bod \[Event Hook \] \(Hook \)](#)

[Vektorová reprezentace \[Embedding \]](#)

[Větev repozitáře \[Repository Branch \] \(Branch \)](#)

[Zapojení člověka do smyčky \[Human-in-the-loop \] \(HITL \)](#)

A

Agent

[CZ] Softwarový systém řízený jazykovým modelem a vybavený nástroji, který samostatně plánuje, vnímá stav prostředí a provádí víceukrokové akce směřující k dosažení zadaného inženýrského cíle.

[EN] A software system driven by a language model and equipped with tools that independently plans, observes its environment, and performs multi-step actions toward a specified engineering goal.

Agentické inženýrství [Agentic Engineering]

[CZ] Inženýrská disciplína zaměřená na návrh, orchestraci a provoz agentických systémů kolem jazykových modelů, včetně nástrojů, kontextu, prováděcích smyček, bezpečnostních mantinelů a lidského dohledu.

[EN] An engineering discipline focused on designing, orchestrating, and operating agentic systems around language models, including tools, context, execution loops, guardrails, and human oversight.

Agentní harness [Agent Harness]

[CZ] Agentní harness — aplikační a orchestrační vrstva obklopující inferenční jádro modelu, která zajišťuje běhové prostředí nástrojů, dynamickou správu kontextového okna, bezpečnostní mantinely, práci se stavem a deterministické řízení životního cyklu požadavku.

[EN] Agent harness — an application and orchestration layer surrounding a model's inference core that provides the tool runtime, dynamic context-window management, guardrails, state handling, and deterministic control over the request lifecycle.

C

Chatbot

[CZ] Systém založený na jazykovém modelu určený primárně k textové interakci s uživatelem; odpovídá na jednotlivé požadavky, ale sám o sobě nedisponuje autonomní prováděcí smyčkou ani nástroji pro samostatnou modifikaci okolního prostředí.

[EN] A language-model-based system designed primarily for text interaction with a user; it responds to individual requests but does not by itself provide an autonomous execution loop or tools for independently modifying the surrounding environment.

D

Degradace kontextu [Context Rot]

[CZ] Degradace pozornosti a kvality logického uvažování modelu způsobená zaplněním kontextového okna dlouhou historií, šumem nebo vzájemně si konkurujícími informacemi, která vede k přehlížení instrukcí a ztrátě souvislostí.

[EN] Degradation in a model's attention and reasoning quality caused by long, noisy, or internally competing context, leading to missed instructions and loss of relationships between facts.

Dovednosti [Skills]

[CZ] Znovupoužitelné modulární balíčky instrukcí (typicky definovaných v souboru SKILL.md), procedurálních pravidel a volitelných pomocných skriptů či zdrojů, které harness dynamicky načítá do kontextu agenta podle povahy řešeného úkolu.

[EN] Reusable modular packages of instructions (typically defined in a SKILL.md file), procedural rules, and optional helper scripts or resources that a harness dynamically loads into an agent's context for a particular class of task.

G

Generování rozšířené vyhledávání [Retrieval-Augmented Generation] (RAG)

[CZ] Architektura, v níž systém před generováním nebo během něj vyhledá relevantní informace z externího zdroje a vloží je do kontextu modelu, aby výstup mohl být založen na načtených datech.

[EN] An architecture in which a system retrieves relevant information from an external source before or during generation and places it into model context so the output can be grounded in the retrieved data.

Git

[CZ] Distribuovaný systém správy verzí, který uchovává historii projektu, podporuje větvení a slučování změn a umožňuje deterministický návrat k předchozím stavům repozitáře.

[EN] A distributed version-control system that records project history, supports branching and merging, and enables deterministic return to earlier repository states.

GitHub

[CZ] Cloudová platforma pro hosting gitových repozitářů, správu vývojového cyklu pomocí Issues a Pull Requests a automatizaci CI/CD pracovních postupů.

[EN] A platform for hosting Git repositories and coordinating the software-development lifecycle through features such as Issues, Pull Requests, and CI/CD automation.

GitHub Actions (Actions)

[CZ] Automatizační platforma GitHubu, která spouští deklarované workflow a jejich joby v reakci na události repozitáře nebo ruční spuštění.

[EN] GitHub's automation platform for running declared workflows and their jobs in response to repository events or manual dispatch.

I

Inženýrství pracovních grafů [Workflow-graph Engineering] (Graph Engineering)

[CZ] Návrh agentních nebo automatizačních pracovních postupů jako explicitních grafů uzlů, závislostí a přechodů namísto jediné neomezené smyčky.

[EN] The design of agentic or automation workflows as explicit graphs of nodes, dependencies, and transitions rather than as one unconstrained loop.

Inženýrství prováděcí smyčky [Execution-loop Engineering] (Loop Engineering)

[CZ] Návrh a řízení iterativní prováděcí smyčky agenta: stavových přechodů, podmínek ukončení, rozpočtů, opakování, eskalací a vazby mezi rozhodováním modelu a nástroji.

[EN] The design and control of an agent's iterative execution loop, including state transitions, termination conditions, budgets, retries, escalation, and the connection between model decisions and tools.

J

Jazykový model [Large Language Model] (LLM)

[CZ] Velký jazykový model je neuronový model trénovaný nad rozsáhlými textovými daty, který autoregresivně zpracovává a generuje posloupnosti tokenů. V této práci vystupuje jako inferenční kognitivní jádro agentního systému.

[EN] A large language model is a neural model trained on large-scale textual data that autoregressively processes and generates token sequences. In this thesis it serves as the inference-based cognitive core of an agentic system.

K

Kompakce kontextu [Context Compaction] (Compaction)

[CZ] Proces zmenšení aktivního kontextu, typicky shrnutím, výběrem nebo nahrazením starších částí historie kompaktnější reprezentací tak, aby se běh vešel do kontextového okna.

[EN] The process of reducing active context, typically by summarizing, selecting, or replacing older history with a more compact representation so execution remains within the context window.

Kontextové inženýrství [Context Engineering]

[CZ] Systematický návrh, výběr, pořadí a životní cyklus informací zpřístupňovaných modelu v aktivním kontextu, včetně instrukcí, paměti, nástrojových výsledků a externě načtených dat.

[EN] The systematic design, selection, ordering, and lifecycle management of information made available to a model in active context, including instructions, memory, tool results, and externally retrieved data.

Kontextové okno [Context Window]

[CZ] Maximální rozsah tokenové sekvence, kterou model při jednom běhu dokáže zahrnout do aktivního kontextu. Prakticky omezuje součet instrukcí, historie, nástrojových výstupů a dalších dat předávaných modelu.

[EN] The maximum token-sequence span a model can include in active context during one inference run. In practice it limits the combined instructions, history, tool outputs, and other data supplied to the model.

M

Mezipaměť klíčů a hodnot [Key-Value Cache] (KV Cache)

[CZ] Mezipaměť dříve vypočtených vektorů klíčů a hodnot v pozornostních vrstvách transformeru, která při autoregresivním generování omezuje nutnost opakovaně přepočítávat předchozí tokeny.

[EN] A cache of previously computed key and value vectors in transformer attention layers that reduces repeated computation of earlier tokens during autoregressive generation.

Model Context Protocol (MCP)

[CZ] Model Context Protocol — otevřený standard původně navržený společností Anthropic pro standardizovanou komunikaci AI aplikací s externími nástroji, zdroji a daty prostřednictvím zpráv JSON-RPC.

[EN] Model Context Protocol — an open standard originally introduced by Anthropic for standardized communication between AI applications and external tools, resources, and data through JSON-RPC messages.

O

Orientovaný acyklický graf [Directed Acyclic Graph] (DAG)

[CZ] Orientovaný graf bez orientovaného cyklu. V pracovních postupech umožňuje explicitně vyjádřit závislosti mezi kroky a pořadí, které z nich vyplývá.

[EN] A directed graph containing no directed cycle. In workflows it can explicitly represent dependencies among steps and the ordering implied by those dependencies.

P

Požadavek na sloučení [Pull Request]

[CZ] Formální návrh na začlenění změn z jedné větve repozitáře do druhé, který slouží jako místo pro automatizované kontroly, lidskou revizi a diskusi nad navrženými úpravami.

[EN] A formal proposal to integrate changes from one repository branch into another, providing a place for automated checks, human review, and discussion of the proposed changes.

Promptové inženýrství [Prompt Engineering]

[CZ] Inženýrská metodika systematického návrhu, strukturování a optimalizace instrukcí a systémových promptů pro řízení chování a mantinelů jazykového modelu.

[EN] An engineering discipline for systematically designing, structuring, and optimizing instructions and system prompts to guide and constrain language-model behavior.

Průběžná integrace [Continuous Integration] (CI)

[CZ] Vývojová praxe, při níž se změny často integrují a automaticky ověřují sestavením, testy a dalšími kontrolami, aby se integrační chyby odhalily co nejdříve.

[EN] A development practice in which changes are integrated frequently and automatically verified by builds, tests, and other checks so integration failures are detected early.

R

Rozšíření [Plugins]

[CZ] Rozšíření běžící přímo v prostředí harnessu, která rozšiřují jeho exekuční jádro o specializované systémové adaptéry, ovladače nástrojů a deterministické záchytné body.

[EN] Programmatic extension modules running directly in the harness environment that extend its execution core with specialized system adapters, tool drivers, and deterministic hooks.

S

Skript [Script]

[CZ] Soubor nebo posloupnost příkazů určených k automatizovanému vykonání interpretu, shellem nebo jiným běhovým prostředím.

[EN] A file or sequence of commands intended for automated execution by an interpreter, shell, or another runtime.

Sloučení commitů [Commit Squashing] (Squash)

[CZ] Operace, při níž se více po sobě jdoucích commitů nahradí jedním souhrnným commitem, obvykle za účelem zjednodušení historie před integrací změn.

[EN] An operation that replaces multiple consecutive commits with one aggregate commit, commonly to simplify history before integrating changes.

Sloučení větví [Branch Merge] (Merge)

[CZ] Operace správy verzí, která kombinuje změny nebo historii dvou vývojových linií do společného výsledného stavu; konflikty vyžadují explicitní vyřešení.

[EN] A version-control operation that combines changes or history from two lines of development into a common resulting state; conflicts require explicit resolution.

Smyčka ReAct [ReAct Loop] (Agent Loop)

[CZ] Iterativní prováděcí cyklus autonomního agenta založený na vzoru ReAct (Reasoning + Acting), v němž model střídavě uvažuje, volá nástroje a vyhodnocuje pozorování z běhového prostředí.

[EN] An iterative execution cycle of an autonomous agent based on the ReAct pattern (Reasoning + Acting), in which the model alternates between reasoning, tool calls, and evaluation of observations from the runtime environment.

Softwarové inženýrství [Software Engineering]

[CZ] Systematické uplatňování inženýrských principů na specifikaci, návrh, implementaci, ověřování, provoz a údržbu softwarových systémů.

[EN] The systematic application of engineering principles to the specification, design, implementation, verification, operation, and maintenance of software systems.

Softwarový kontejner [Software Container] (Container)

[CZ] Izolované uživatelské běhové prostředí balící aplikaci a její závislosti při sdílení jádra hostitelského operačního systému; úroveň bezpečnostní izolace závisí na konkrétní implementaci a konfiguraci.

[EN] An isolated user-space runtime packaging an application and its dependencies while sharing the host operating-system kernel; its security isolation depends on the implementation and configuration.

Správa verzí [Version control]

[CZ] Správa a sledování změn zdrojových souborů a dalších verzovaných artefaktů tak, aby bylo možné změny bezpečně větvit, slučovat, auditovat a v případě potřeby vracet.

[EN] The management and tracking of changes to source files and other versioned artifacts so changes can be safely branched, merged, audited, and reverted when necessary.

T

Tah interakce [Interaction Turn] (Turn)

[CZ] Jedna diskrétní jednotka interakce v konverzačním nebo agentním protokolu, například zpráva uživatele, odpověď modelu nebo samostatně evidovaný výsledek nástroje.

[EN] One discrete unit of interaction in a conversational or agentic protocol, such as a user message, model response, or separately recorded tool result.

Token

[CZ] Diskrétní jednotka zpracovávaná jazykovým modelem. Token odpovídá položce slovníku tokenizéru a je reprezentován číselným identifikátorem; nemusí odpovídat celému slovu.

[EN] A discrete unit processed by a language model. A token corresponds to an entry in the tokenizer vocabulary and is represented by a numeric identifier; it need not correspond to a whole word.

Tokenizér [Tokenizer]

[CZ] Komponenta, která převádí text nebo jiný vstup na posloupnost tokenů a jejich identifikátorů a podle podporovaného směru také provádí zpětnou dekodovací transformaci.

[EN] A component that maps text or another input into a sequence of tokens and token identifiers and, where supported, performs the reverse decoding transformation.

Transformerová architektura [Transformer Architecture] (Transformer)

[CZ] Architektura neuronových sítí založená na mechanismu pozornosti, která modeluje vztahy mezi prvky sekvence a tvoří základ většiny současných velkých jazykových modelů.

[EN] A neural-network architecture based on attention mechanisms that models relationships among sequence elements and underlies most contemporary large language models.

U

Událostní záchytný bod [Event Hook] (Hook)

[CZ] Definovaný bod životního cyklu nebo události, na který lze navázat vlastní deterministickou logiku před, po nebo místo standardního chování systému.

[EN] A defined lifecycle or event point to which custom deterministic logic can be attached before, after, or in place of standard system behavior.

V

Vektorová reprezentace [Embedding]

[CZ] Vícerozměrná vektorová reprezentace tokenů nebo jiných dat, v níž numerické vztahy mezi vektory zachycují užitečné sémantické vztahy mezi reprezentacemi.

[EN] A multidimensional vector representation of tokens or other data in which numerical relationships between vectors capture useful semantic relationships between representations.

Větev repozitáře [Repository Branch] (Branch)

[CZ] Pojmenovaná vývojová linie v systému správy verzí, která umožňuje provádět změny odděleně od jiné linie historie a později je porovnat nebo sloučit.

[EN] A named line of development in version control that allows changes to proceed separately from another history line and later be compared or merged.

Z

Zapojení člověka do smyčky [Human-in-the-loop] (HITL)

[CZ] Návrhový vzor, v němž lidský operátor zůstává součástí rozhodovacího procesu systému prostřednictvím schvalovacích bran (Human Gates), zejména před významnými nebo nevratnými systémovými operacemi.

[EN] A design pattern in which a human operator remains part of the system's decision process through approval gates (Human Gates), especially before consequential or irreversible system operations.

Seznam příloh | List of appendices

Seznam zdrojů | References

1. CHACON, Scott a STRAUB, Ben. *Pro Git*. 2. New York : Apress, 2014. IS-BN 978-1-4842-0076-6.
2. VASWANI, Ashish, SHAZEER, Noam, PARMAR, Niki, USZKOREIT, Jakob, JONES, Llion, GOMEZ, Aidan N., KAISER, Łukasz a POLOSUKHIN, Illia. Attention Is All You Need. *arXiv preprint arXiv:1706.03762*. Online. 2017. Available from: <https://arxiv.org/abs/1706.03762>
3. YAO, Shunyu, ZHAO, Jeffrey, YU, Dian, DU, Nan, SHAFRAN, Izhak, NARASIMHAN, Karthik a CAO, Yuan. ReAct: Synergizing Reasoning and Acting in Language Models. *arXiv preprint arXiv:2210.03629*. Online. 2022. Available from: <https://arxiv.org/abs/2210.03629>
4. MARIUS, Patrik. DarkFactory: autonomous, governed software engineering pipelines. Online. 2026. [Accessed 8 září 2026]. Available from: <https://github.com/marius-patrik/DarkFactory>
5. HUMBLE, Jez a FARLEY, David. *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Boston : Addison-Wesley, 2010. ISBN 978-0-321-60191-9.
6. Deepseek Harness. *arXiv preprint arXiv:2608.25512*. Online. 2026. Available from: <https://arxiv.org/abs/2608.25512>
7. ANTHROPIC. Model Context Protocol documentation. Online. 2026. Available from: <https://modelcontextprotocol.io/docs/2026-07-28/getting-started/intro#explore-mcp>
8. ANTHROPIC. Prompt Engineering overview. Online. 2026. Available from: <https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/overview>
9. LIU, Nelson F., LIN, Kevin, HEWITT, John, PARANJAPE, Ashwin, BEVILACQUA, Michele, PETRONI, Fabio a LIANG, Percy. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics*. Online. 2024. Vol. 12, p. 157–173. Available from: <https://arxiv.org/abs/2307.03172>
10. DAO, Tri, FU, Daniel Y., ERMON, Stefano, RUDRA, Atri a RÉ, Christopher. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. *Advances in Neural Information Processing Systems*. Online. 2022. Vol. 35, p. 16344–16359. Available from: <https://arxiv.org/abs/2205.14135>
11. AINSLIE, Joshua, LEE-THORP, James, JONG, Michiel de, ZEMLYANSKIY, Yury, LEBRÓN, Federico a SANGHAI, Sumit. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. *arXiv preprint arXiv:2305.13245*. Online. 2023. Available from: <https://arxiv.org/abs/2305.13245>